

UNIVERSIDAD DE MANIZALES

Tecnología en desarrollo de software para negocios digitales

Seguridad de aplicaciones

Universidad de manizales

Sergio Pinilla Valencia

Pablo Nieto Pérez

Negocioseguro (compra segura)

Tarea 6, migración criptográfica del proyecto negocioseguro

Evidencia de migración criptográfica

6 de noviembre de 2026

1. Introducción

Desde la auditoría exploratoria de la tarea 1 quedó anotado que lo más grave de negocioseguro era que las contraseñas estaban guardadas en texto plano, en el producto 2 tapé la inyección sql pero en las limitaciones volví a dejar escrito que las claves seguían a la vista y que eso iba en otra entrega, y en la tarea 5 probé por aparte cómo funcionan password_hash y openssl_encrypt, esta tarea es donde todo eso se mete por fin en la aplicación

Seguí la guía del estudiante de la clase 9 en sus tres fases: primero pasé las contraseñas a hash (las nuevas y las que ya estaban guardadas), después cifré la dirección de envío con aes-256-cbc dejando la clave de cifrado en una variable de entorno de windows y no en el código, y al final corrí las cuatro pruebas obligatorias, todo en mi xampp local y con datos inventados

El documento tiene lo que hice en cada fase con sus capturas, la plantilla de evidencia diligenciada, la comparación de la base de datos antes y después, la reflexión, las tres preguntas de análisis y en los anexos el código de los tres archivos que van en el zip

2. Objetivos

2.1 Objetivo general

Proteger las contraseñas y las direcciones de envío de negocioseguro con hashing y cifrado simétrico sin que la aplicación deje de funcionar para los usuarios que ya existían

2.2 Objetivos específicos

1. Ampliar la columna clave y crear la columna direccion para que la tabla usuarios pueda guardar hashes y datos cifrados

2. Cambiar registro.php para que guarde la contraseña con password_hash y la dirección cifrada, y login.php para que valide con password_verify

3. Migrar con migrar_claves.php las contraseñas que ya estaban en texto plano sin dañar las que ya estuvieran migradas

4. Dejar la clave de cifrado en una variable de entorno del sistema operativo, por fuera del código php

5. Comprobar con las cuatro pruebas obligatorias que el registro, el login y el descifrado siguen funcionando

3. Desarrollo

3.1 Preparación y respaldo

El proyecto lo tengo en c:\xampp\htdocs\compraseguras, entonces donde la guía dice http://localhost/NegocioSeguro/ en mi caso es http://localhost:8080/compraseguras/ porque mi apache escucha en el puerto 8080, el xampp trae php 8.2.12 y mariadb, prendí apache y mysql

y confirmé lo que pide la guía antes de empezar: que existan conexion.php, registro.php y login.php con las consultas preparadas del producto 2 y que la tabla usuarios tenga id, usuario y clave

Antes de tocar nada saqué un respaldo de la base con mysqldump (compraseguras_antes_tarea6.sql) y lo guardé por fuera de la carpeta htdocs, después tomé las capturas del estado inicial: los dos usuarios del archivo compraseguras.sql con la clave a la vista y la columna clave como varchar(50)



Figura 1, tabla usuarios antes de la actividad, admin123 y miclave2024 se leen tal cual

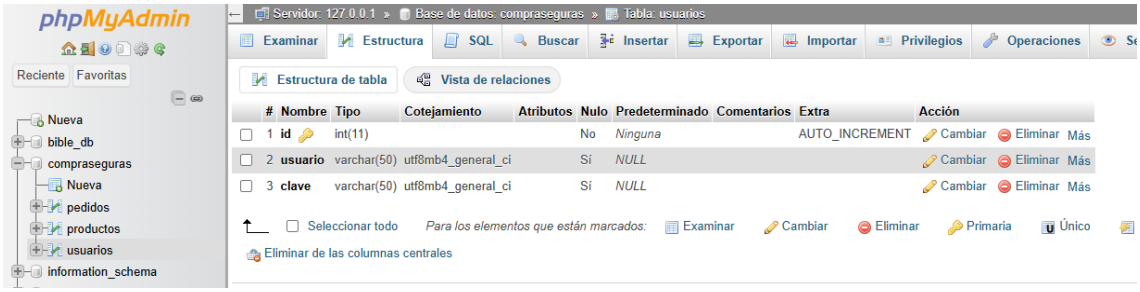


Figura 2, estructura antes de la actividad, clave es varchar(50) y no hay columna para la dirección

3.2 Fase 1, migración de contraseñas

Paso 1, en la pestaña sql de phpmyadmin corrí ALTER TABLE usuarios MODIFY COLUMN clave VARCHAR(255); y en la estructura la columna quedó de 255, eso ya lo había dicho en la tarea 1: un hash de bcrypt tiene 60 caracteres y no cabe en 50

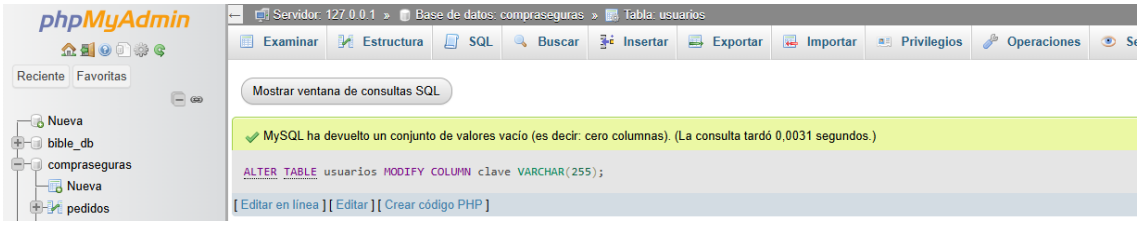


Figura 3, el alter de la columna clave ejecutado en phpmyadmin

Pasos 2 y 3, en registro.php la contraseña ahora pasa por password_hash($claveTexto, PASSWORD_DEFAULT) antes del insert y lo que se guarda es $claveHash, nunca lo que

escribió el usuario, y en login.php cambié la comparación $clave === $fila['clave'] por password_verify, que saca la sal y el costo del mismo hash, conservé la consulta preparada del producto 2 para buscar al usuario

Al login le agregué tres cosas que no estaban en el código de la guía: el mismo mensaje "Usuario o clave incorrectos" tanto si el usuario no existe como si la clave está mal, para no dar pistas de qué usuarios existen, session_regenerate_id al entrar para que no se pueda reutilizar un id de sesión de antes del login, y password_needs_rehash, que si algún día php sube el costo o cambia el algoritmo por defecto renueva el hash del usuario la próxima vez que entra, sin tener que hacer otra migración

Paso 4, migrar_claves.php es el script de la guía: recorre la tabla, si la clave ya empieza por $2y$ la cuenta como migrada y la salta, y si no le hace password_hash y la actualiza con una consulta preparada, le cambié dos cosas, preparo el update una sola vez por fuera del ciclo y al principio el script revisa que la petición venga del mismo equipo (127.0.0.1), porque un archivo así en una carpeta pública es una puerta abierta mientras exista

Paso 5, lo abrí en el navegador y migró las dos contraseñas, después lo volví a correr para comprobar que la condición del $2y$ funciona y efectivamente no tocó nada la segunda vez, eso es importante porque si volviera a hashear un hash el usuario ya no podría entrar con su clave

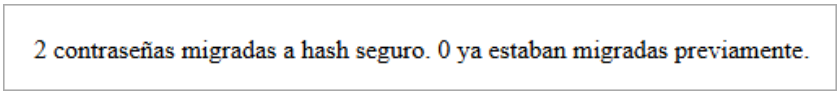


Figura 4, primera ejecución de migrar_claves.php

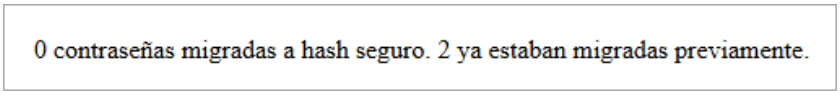


Figura 5, segunda ejecución, el script es idempotente y no vuelve a procesar los hashes

En phpmyadmin las dos claves quedaron ilegibles, con 60 caracteres y empezando por $2y$10$ (bcrypt con costo 10, que es lo que usa PASSWORD_DEFAULT en php 8.2), y aunque admin y jperez se migraron el mismo segundo cada hash es distinto porque cada uno lleva su propia sal, desde la consola de mysql lo confirmé con LENGTH(clave):

id	usuario	prefijo	largo
1	admin	$2y$10$	60
2	jperez	$2y$10$	60



Figura 6, las claves de admin y jperez después de la migración

3.3 Fase 2, cifrado de la dirección de envío

Paso 6, agregué la columna con ALTER TABLE usuarios ADD COLUMN direccion TEXT; esta la corrí desde la consola de mysql de xampp y no desde phpmyadmin, el resultado es el mismo y se ve en la estructura final de la sección 4

Paso 7, la clave de cifrado. La guía propone UnaClaveDe32CaracteresExactosOK pero le saqué el largo y tiene 31 caracteres, igual que me pasó en la tarea 5 con la clave del código base, y openssl no avisa nada, rellena la clave por dentro y sigue, entonces en vez de esa generé una de 32 caracteres aleatorios (letras y números) y la guardé con setx NEGOCIOSEGURO_CLAVE_CIFRADO "..." en una variable de entorno de mi usuario de windows, la consola respondió "CORRECTO: se guardó el valor especificado", reinicié apache para que la tomara y comprobé con un archivo de prueba que php la veía con largo 32, ese archivo lo borré enseguida

La clave no aparece en ningún archivo del proyecto ni en este documento, en el código solo está el nombre de la variable, getenv('NEGOCIOSEGURO_CLAVE_CIFRADO'), así que si alguien ve el código o el repositorio no se lleva la llave

Paso 8, en registro.php agregué el campo "Dirección de envío" y el bloque de la guía: un iv aleatorio de 16 bytes por cada registro, openssl_encrypt con aes-256-cbc, el iv pegado adelante del texto cifrado y todo en base64 en la columna direccion, en el mismo insert que ya guardaba el hash de la contraseña

Acá encontré algo que el código de la guía no controla: si la variable de entorno no existe, getenv devuelve false y openssl_encrypt igual cifra, con una clave vacía rellena de ceros, lo probé en la consola y cifró y descifró "Calle 1" sin ningún error, o sea que si apache arranca sin la variable las direcciones quedarían "cifradas" con una clave que cualquiera puede adivinar y nadie se daría cuenta, por eso registro.php revisa que la clave exista y tenga 32 caracteres y si no, no registra a nadie, también valida que la dirección no venga vacía ni pase de 150 caracteres

Paso 9, verificar_direccion.php es el de la guía con un cambio en la consulta: la guía hace SELECT usuario, direccion FROM usuarios LIMIT 1, que trae al primer usuario, y el primero es admin, que se registró antes de que existiera la columna y tiene la dirección en NULL, entonces la prueba saldría vacía, la cambié para que traiga el último usuario que sí tiene dirección y además muestra lo que está guardado en la base al lado de lo descifrado, también quedó restringido a 127.0.0.1 igual que migrar_claves.php

3.4 Fase 3, pruebas de verificación

Paso 10 y 11, con todo listo corrí las cuatro pruebas obligatorias desde el navegador, más una quinta que no pedía la guía: entrar con una clave equivocada, para estar seguro de que password_verify no deja pasar cualquier cosa

Prueba 1, registro nuevo: registré a laura_gomez (el usuario inventado de la tarea 5) con una clave de 17 caracteres y la dirección Carrera 23 # 65-10, barrio Palermo, Manizales

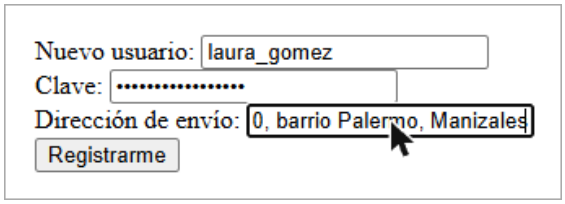


Figura 7, formulario de registro con el campo nuevo de dirección

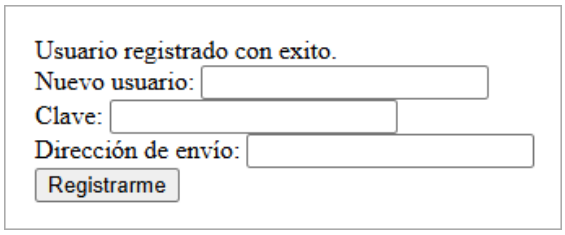


Figura 8, respuesta del registro

Prueba 2, login nuevo: entré con laura_gomez y su clave

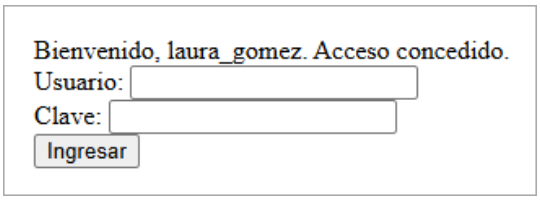


Figura 9, login con el usuario recién creado

Prueba 3, login anterior: entré con jperez y miclave2024, que es su clave de siempre del archivo compraseguras.sql, o sea un usuario que se creó en texto plano y que pasó por la migración, y con miclave2025 el sistema lo rechaza

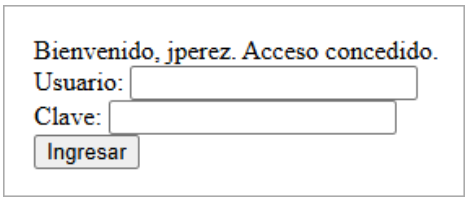


Figura 10, login de jperez con su contraseña original después de la migración

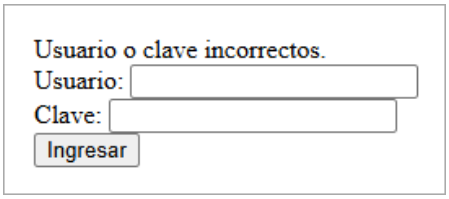


Figura 11, prueba extra, la misma cuenta con una clave equivocada

Prueba 4, descifrado: abrí verificar_direccion.php, en la base la dirección es un base64 de 108 caracteres que no se parece en nada al texto y descifrada sale exactamente la que escribí en el registro

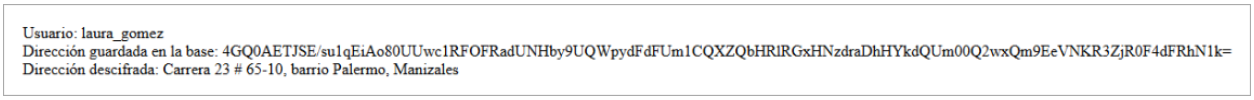


Figura 12, verificar_direccion.php, lo guardado en la base y lo descifrado

4. Evidencia de migración criptográfica — negocioseguro

Equipo: trabajo individual
Integrantes: Pablo Nieto Pérez
Fecha: 6 de noviembre de 2026

Prueba	Resultado esperado	Resultado obtenido	¿Coincide?
Registro de usuario nuevo	Contraseña hasheada y dirección cifrada	laura_gomez quedó con la clave como hash $2y$10$ de 60 caracteres y la dirección como base64 ilegible de 108 caracteres, el registro respondió "Usuario registrado con exito"	Sí
Login con usuario nuevo	Acceso concedido	"Bienvenido, laura_gomez. Acceso concedido."	Sí
Login con usuario anterior	Acceso concedido	jperez entró con miclave2024, su clave de antes de la migración: "Bienvenido, jperez. Acceso concedido.", con una clave equivocada respondió "Usuario o clave incorrectos."	Sí
Descifrado de dirección	Coincide con el dato original	verificar_direccion.php devolvió Carrera 23 # 65-10, barrio Palermo, Manizales, igual a lo escrito en el registro	Sí

4.1 Comparación de la base de datos

Antes la tabla usuarios tenía tres columnas, clave era varchar(50) y cualquiera que entrara a phpmyadmin leía admin123 y miclave2024 tal cual (figuras 1 y 2), ahora clave es varchar(255) y hay una cuarta columna direccion de tipo text, las tres contraseñas son hashes de 60 caracteres que empiezan por $2y$10$ y son distintos entre sí, y la dirección de laura_gomez es un texto en base64 que no deja ver nada de la dirección real, admin y jperez tienen la dirección en NULL porque se crearon antes de que existiera el campo y no había ninguna dirección que cifrar



Figura 13, estructura después de la actividad, clave varchar(255) y la columna nueva direccion

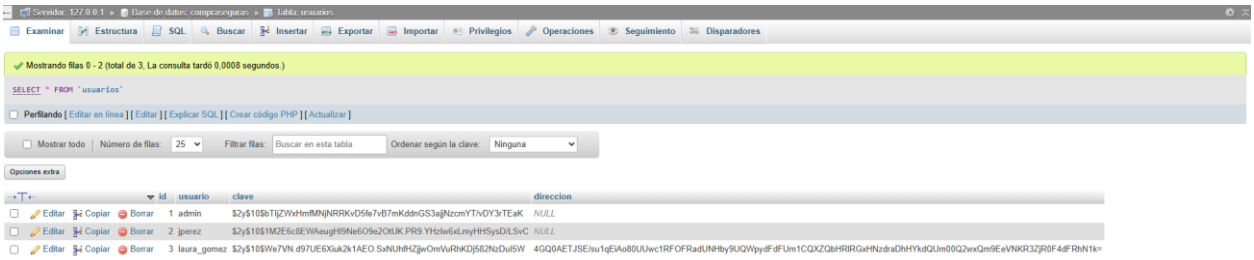


Figura 14, tabla usuarios al final, contraseñas en hash y la dirección cifrada

Algo que no se ve en la tabla pero también cambió: con el respaldo de antes (o con el archivo compraseguras.sql original) se pueden leer las claves viejas, por eso el respaldo quedó guardado por fuera de la carpeta pública y lo voy a tratar con el mismo cuidado que la clave de cifrado

4.2 Reflexión final

Antes, si alguien lograba ver la base de datos de la tienda se llevaba las contraseñas de todos los clientes listas para usar, desde hoy las contraseñas se guardan como una huella que sirve para comprobar que la clave es la correcta pero que no se puede devolver a la clave original, y las direcciones quedan bajo candado con una llave que se guarda aparte del sistema, los clientes siguen entrando con la misma contraseña de siempre y no notan ningún cambio

4.3 Preguntas de análisis

1. ¿Qué hallazgo de seguridad documentado desde la primera clase queda cerrado con esta actividad?

El de las contraseñas en texto plano, en la auditoría exploratoria de la tarea 1 lo puse como el punto más crítico de la base de datos con nivel de preocupación alto: cualquiera que abriera compraseguras.sql o entrara a phpmyadmin veía las claves de todos, y en el producto 2 lo dejé en las limitaciones como la categoría a02 del owasp (fallas criptográficas) que faltaba, con esta tarea queda cerrado: en la tabla ya no hay ninguna contraseña legible, las nuevas entran como hash desde el registro y las viejas se migraron, y de paso la dirección de envío, que es un dato personal, ya tampoco se guarda en claro

2. ¿Por qué fue necesario modificar la estructura de la base de datos y no solamente el código PHP?

Porque el dato protegido no cabe donde estaba el dato original, un hash de bcrypt mide 60 caracteres y la columna clave era de 50, si solo cambiaba el php el update de la migración y el insert del registro fallaban por "data too long" o, en un servidor sin modo estricto, el hash se guardaba cortado y nadie podía volver a entrar, eso ya lo había anotado en la tarea 1, se dejó en 255 y no en 60 porque php recomienda ese tamaño por si PASSWORD_DEFAULT pasa a un algoritmo con hashes más largos como argon2

Con la dirección pasa algo parecido: no había columna para guardarla y lo cifrado ocupa más que el texto original, por el iv de 16 bytes, el relleno de aes y el base64, la dirección de 45

caracteres de laura_gomez quedó de 108, por eso la columna es text, la base de datos y el código tienen que cambiar juntos, si no el código seguro no tiene dónde guardar lo que produce

3. ¿Por qué se utilizó hashing para las contraseñas y cifrado para las direcciones?

Por lo que la tienda necesita hacer con cada dato, la contraseña nunca hay que volver a leerla, solo comprobar si lo que escribe el usuario es lo mismo, para eso sirve un hash de una sola vía y lento como bcrypt: password_verify compara sin necesitar la clave original y si se roban la tabla no hay una llave que las devuelva, solo queda probar clave por clave y cada intento cuesta (en la tarea 5 medí unos 220 milisegundos por hash con costo 12)

La dirección sí hay que leerla, sin ella no se puede despachar un pedido, entonces tiene que ser reversible y por eso va con cifrado simétrico aes-256: quien tiene la clave la recupera completa y quien solo ve la base no ve nada, si se hubiera hecho al revés no funcionaría ninguna de las dos cosas, con la dirección hasheada la tienda no podría enviar nada y con las contraseñas cifradas bastaría con robar la clave de cifrado para leer las de todos los usuarios

5. Cierre y limitaciones

Como pide la guía, después de las pruebas borré migrar_claves.php y verificar_direccion.php de la carpeta pública del proyecto, las copias que van en la entrega quedaron por fuera de htdocs junto con el respaldo de la base, y la clave de cifrado solo existe en la variable de entorno de windows

Lo que no queda resuelto y es mejor decirlo:

• aes-256-cbc protege que la dirección no se lea pero no detecta si alguien con acceso a la base cambia el texto cifrado, para eso sería mejor aes-256-gcm, que trae una etiqueta de autenticación, o agregarle un hmac

• La variable de entorno la puede leer cualquier programa que corra con mi usuario de windows, en producción la clave debería ir en un gestor de secretos o en una variable que solo vea el servicio de la aplicación, y no se cubrió cómo rotarla (cambiar la clave obliga a descifrar y volver a cifrar todas las direcciones)

• La versión en línea (negocioseguro.wib.digital) todavía no tiene estos cambios, para subirlos hay que correr los dos alter en su base, definir la variable NEGOCIOSEGURO_CLAVE_CIFRADO en el contenedor y ejecutar la migración allá también

• Las claves débiles siguen siendo débiles, admin123 ahora es un hash pero sigue siendo fácil de adivinar probando una lista de claves comunes, falta una política de contraseñas y limitar los intentos de login

6. Bibliografía

Owasp foundation, (s.f.), password storage cheat sheet, recuperado el 6 de noviembre de 2026 de https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Owasp foundation, (s.f.), cryptographic storage cheat sheet, recuperado el 6 de noviembre de 2026 de https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

Owasp foundation, (2021), a02:2021 cryptographic failures, recuperado el 6 de noviembre de 2026 de https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

The php group, (s.f.), password_hash, manual de php, recuperado el 6 de noviembre de 2026 de https://www.php.net/manual/es/function.password-hash.php

The php group, (s.f.), password_verify y password_needs_rehash, manual de php, recuperado el 6 de noviembre de 2026 de https://www.php.net/manual/es/function.password-verify.php

The php group, (s.f.), openssl_encrypt, manual de php, recuperado el 6 de noviembre de 2026 de https://www.php.net/manual/es/function.openssl-encrypt.php

Pinilla valencia, s, (2026), guía para el estudiante, migración criptográfica del proyecto negocioseguro, clase 9, hashing en código, universidad de manizales

7. Anexos

Anexo a, código de registro.php, también va en el zip de la entrega

```
<?php
include 'conexion.php';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $usuario    = trim($_POST['usuario'] ?? '');
    $claveTexto = $_POST['clave'] ?? '';
    $direccion  = trim($_POST['direccion'] ?? '');

    if (!preg_match('/^[a-zA-Z0-9_]{3,20}$/', $usuario)) {
        die("Nombre de usuario invalido. Use solo letras, numeros y guion bajo (3 a 20
caracteres).");
    }

    if (strlen($claveTexto) < 8) {
        die("La clave debe tener al menos 8 caracteres.");
    }

    if ($direccion === '' || mb_strlen($direccion) > 150) {
        die("La direccion de envio es obligatoria (maximo 150 caracteres).");
    }

    // La clave de cifrado vive en una variable de entorno del sistema, no en el codigo
    $claveCifrado = getenv('NEGOCIOSEGURO_CLAVE_CIFRADO');
    if ($claveCifrado === false || strlen($claveCifrado) !== 32) {
        die("El registro no esta disponible en este momento.");
    }

    // La contrasena se guarda como hash, nunca como texto
    $claveHash = password_hash($claveTexto, PASSWORD_DEFAULT);

    // La direccion se cifra con AES-256-CBC y un IV aleatorio por usuario
    $metodo   = "AES-256-CBC";
    $ivLength = openssl_cipher_iv_length($metodo);
    $iv       = openssl_random_pseudo_bytes($ivLength);

    $direccionCifrada = openssl_encrypt($direccion, $metodo, $claveCifrado, 0, $iv);
    if ($direccionCifrada === false) {
        die("No se pudo registrar el usuario.");
    }

    // Guardar IV y texto cifrado juntos
    $direccionParaGuardar = base64_encode($iv . $direccionCifrada);

    $stmt = $conn->prepare("INSERT INTO usuarios (usuario, clave, direccion) VALUES (?, ?, ?)");
    $stmt->bind_param("sss", $usuario, $claveHash, $direccionParaGuardar);

    if ($stmt->execute()) {
        echo "Usuario registrado con exito.";
    } else {
        echo "No se pudo registrar el usuario.";
    }
    $stmt->close();
}
?>
<form method="POST">
  Nuevo usuario: <input type="text" name="usuario"><br>
  Clave: <input type="password" name="clave"><br>
  Dirección de envío: <input type="text" name="direccion"><br>
```

```
  <button type="submit">Registrarme</button>
</form>
```

Anexo b, código de login.php, también va en el zip de la entrega

```
<?php
include 'conexion.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $usuario        = $_POST['usuario'] ?? '';
    $claveIngresada = $_POST['clave'] ?? '';

    $stmt = $conn->prepare("SELECT id, usuario, clave FROM usuarios WHERE usuario = ?");
    $stmt->bind_param("s", $usuario);
    $stmt->execute();
    $resultado = $stmt->get_result();
    $fila = $resultado ? $resultado->fetch_assoc() : null;
    $stmt->close();

    // password_verify saca la sal y el costo del propio hash y compara de forma segura
    if ($fila && password_verify($claveIngresada, $fila['clave'])) {
        session_regenerate_id(true);
        $_SESSION['usuario_id'] = $fila['id'];
        $_SESSION['usuario']    = $fila['usuario'];

        // Si PHP cambia el algoritmo o el costo por defecto, el hash se renueva en este login
        if (password_needs_rehash($fila['clave'], PASSWORD_DEFAULT)) {
            $nuevoHash  = password_hash($claveIngresada, PASSWORD_DEFAULT);
            $stmtUpdate = $conn->prepare("UPDATE usuarios SET clave = ? WHERE id = ?");
            $stmtUpdate->bind_param("si", $nuevoHash, $fila['id']);
            $stmtUpdate->execute();
            $stmtUpdate->close();
        }

        echo "Bienvenido, " . htmlspecialchars($fila['usuario']) . ". Acceso concedido.";
    } else {
        echo "Usuario o clave incorrectos.";
    }
}
?>
<form method="POST">
  Usuario: <input type="text" name="usuario"><br>
  Clave: <input type="password" name="clave"><br>
  <button type="submit">Ingresar</button>
</form>
```

Anexo c, código de migrar_claves.php, también va en el zip de la entrega, ya borrado de la carpeta pública

```
<?php
// Script de una sola vez: pasa a hash las claves que siguen en texto plano.
// Solo se puede abrir desde el mismo equipo, y se borra de la carpeta publica al terminar.
$origen = $_SERVER['REMOTE_ADDR'] ?? '';
if (PHP_SAPI !== 'cli' && !in_array($origen, ['127.0.0.1', '::1'], true)) {
    http_response_code(403);
    die("Acceso denegado.");
}

include 'conexion.php';
```

```
$resultado = $conn->query("SELECT id, clave FROM usuarios");

$migrados   = 0;
$yaMigrados = 0;

$stmtUpdate = $conn->prepare("UPDATE usuarios SET clave = ? WHERE id = ?");

while ($fila = $resultado->fetch_assoc()) {
    // Evitar volver a procesar contrasenas ya migradas (bcrypt empieza por $2y$)
    if (strpos($fila['clave'], '$2y$') === 0) {
        $yaMigrados++;
        continue;
    }

    $nuevoHash = password_hash($fila['clave'], PASSWORD_DEFAULT);

    $stmtUpdate->bind_param("si", $nuevoHash, $fila['id']);
    $stmtUpdate->execute();
    $migrados++;
}
$stmtUpdate->close();

echo "$migrados contraseñas migradas a hash seguro. ";
echo "$yaMigrados ya estaban migradas previamente.";
?>
```

Anexo d, código de verificar_direccion.php, script de comprobación, no va en el zip, ya borrado de la carpeta pública

```
<?php
// Archivo temporal de comprobacion: se borra de la carpeta publica al terminar las pruebas.
$origen = $_SERVER['REMOTE_ADDR'] ?? '';
if (PHP_SAPI !== 'cli' && !in_array($origen, ['127.0.0.1', '::1'], true)) {
    http_response_code(403);
    die("Acceso denegado.");
}

include 'conexion.php';

$resultado = $conn->query("SELECT usuario, direccion FROM usuarios WHERE direccion IS NOT NULL ORDER
BY id DESC LIMIT 1");
$fila = $resultado->fetch_assoc();
if (!$fila) {
    die("No hay usuarios con direccion registrada.");
}

$claveCifrado = getenv('NEGOCIOSEGURO_CLAVE_CIFRADO');
$metodo       = "AES-256-CBC";
$ivLength     = openssl_cipher_iv_length($metodo);

$datosGuardados   = base64_decode($fila['direccion']);
$iv               = substr($datosGuardados, 0, $ivLength);
$direccionCifrada = substr($datosGuardados, $ivLength);

$direccionOriginal = openssl_decrypt($direccionCifrada, $metodo, $claveCifrado, 0, $iv);

echo "Usuario: " . htmlspecialchars($fila['usuario']) . "<br>";
echo "Dirección guardada en la base: " . htmlspecialchars($fila['direccion']) . "<br>";
```

```
echo "Dirección descifrada: " . ($direccionOriginal === false ? "no se pudo descifrar" :
htmlspecialchars($direccionOriginal)) . "<br>";
?>
```